

Correction détaillée — Chaînes de caractères (approfondissement)**Exercice 1**

```
1 identite = "Martin Durand-Dupont"
2 init = identite[0]
3 for i in range(len(identite)):
4     if identite[i] == ' ':
5         init = init + identite[i+1]
6 print(init.upper())
```

► **1. Ligne 2.** `init = identite[0]` place dans `init` le **premier caractère** de la chaîne (ici la première lettre du nom).

► **2. La boucle bornée** parcourt la chaîne caractère par caractère ; chaque fois qu'elle rencontre une **espace**, elle ajoute à `init` le caractère *suivant* (`identite[i+1]`), c'est-à-dire la première lettre du mot qui suit. La fonction construit ainsi les **initiales**.

► **3. Ligne 6.** `print(init.upper())` affiche `init` en le convertissant en **majuscules** (méthode `upper()`).

► **4. Affichages.**

- **a.** « martin durand » : initiale du 1^{er} mot (m) + lettre après l'espace (d) → MD.
- **b.** « martin dupont durant » : deux espaces → m + d + d → MDD.
- **c.** « martin dupont-durant » : une *seule* espace (le trait d'union n'est pas une espace) → m + d → MD.

► **5.** Pour que 4.c donne le même résultat que 4.b (MDD), il faut traiter le **trait d'union comme un séparateur**, en ajoutant un test sur `'-'` :

```
1     if identite[i] == ' ' or identite[i] == '-':
2         init = init + identite[i+1]
```

Désormais « martin dupont-durant » donne aussi MDD.

Exercice 2

```
1 phrase = "Donner une expression"
2 affiche = phrase[0]
3 print(affiche)
4 for i in range(1, len(phrase)):
5     affiche = affiche + phrase[i]
6     print(affiche)
```

- 1. Ligne 2. `affiche = phrase[0]` place dans `affiche` le **premier caractère** de la chaîne.
- 2. La **boucle bornée** ajoute à `affiche`, l'un après l'autre, les caractères suivants de la chaîne (de l'indice 1 jusqu'à la fin) et **affiche à chaque étape** la chaîne en cours de construction. On voit donc la chaîne « grandir » lettre par lettre.
- 3. Pour « Stylé », la console affiche successivement :

```
S
St
Sty
Styl
Stylé
```

- 4. Avec `range(len(phrase))` au lieu de `range(1, len(phrase))`, la boucle commence à l'indice 0 : dès le premier tour, on ajoute *de nouveau* `phrase[0]` à `affiche` (qui le contenait déjà). La **première lettre est donc dupliquée** :

```
S
SS
SSt
SSty
SStyl
SStylé
```

Exercice 3

```
1 def essai(n):
2     n = str(n)
3     return(len(n))
```

- 1. `str(n)` convertit `n` en **chaîne de caractères**. Cela permet d'utiliser `len()` pour *compter les caractères*, même si `n` est un nombre : sans cette conversion, `len()` appliqué à un entier provoquerait une erreur.
- 2. Valeurs renvoyées.
 - a. `essai(123)` : `str(123) = "123"`, qui a 3 caractères → **3**.
 - b. `essai('Bonjour')` : la chaîne "Bonjour" a 7 caractères → **7**.
 - c. `essai(-596)` : `str(-596) = "-596"` a 4 caractères, car le **signe « - » compte** comme un caractère → **4** (et non 3).

Exercice 4 — Le code ASCII

► 1. Affichages.

<code>ord('A')</code>	65
<code>ord('Z')</code>	90
<code>chr(65)</code>	'A'

► 2. La fonction `ord()` renvoie le **code ASCII** (un entier) d'un caractère ; la fonction `chr()` fait l'**opération inverse** : elle renvoie le caractère correspondant à un code ASCII. Ce sont deux fonctions réciproques l'une de l'autre.

Exercice 5 — `upper`, `lower`, `capitalize`, `\t`, `\n`► 1. Affichages (avec `phrase = 'La vie est belle'`).

- a. `phrase.upper()` → 'LA VIE EST BELLE'
- b. `phrase.lower()` → 'la vie est belle'
- c. `phrase.capitalize()` → 'La vie est belle'

► 2. Rôles. `upper()` met tout en **majuscules** ; `lower()` met tout en **minuscules** ; `capitalize()` met **seulement la première lettre** en majuscule et le reste en minuscules.

► 3. Script complété. Le nom doit être en majuscules (`upper`) et le prénom avec une initiale majuscule (`capitalize`) :

```
1 def identite(nom, prenom, age):
2     nom = nom.upper()
3     prenom = prenom.capitalize()
4     age = str(age)
5     print("Salut", prenom, nom, "\ntu as", age, "ans")
```

Par exemple, `identite("durand", "martin", 16)` affiche :

```
Salut Martin DURAND
tu as 16 ans
```

► 4. `\t` insère une **tabulation** (un espacement horizontal) et `\n` provoque un **retour à la ligne**.

Exercice 6 — Le chiffrement de César

- **1.a. Tableau complété** (décalage de +3, codes ASCII).

Lettre en clair	C	E	S	A	R
Code ASCII	67	69	83	65	82
Code ASCII + 3	70	72	86	68	85
Lettre codée	F	H	V	D	U

- **1.b.** Utiliser le code César revient à ajouter **3** au code ASCII des lettres en clair du message.

- **2.** Le code ASCII de l'espace est `ord(' ') = 32`.

- **3.a. La boucle bornée** parcourt chaque caractère du message (mis en majuscules) ; pour chacun, elle calcule son code ASCII (`ord`), lui ajoute 3, reconvertit le résultat en caractère (`chr`) et l'ajoute à la chaîne codée.

- **3.b. Codages obtenus.**

- CESAR → FHVDU (correct)
- ROYAUME → UR\DXPH (le Y devient « \ »)
- bizarre → BIZARRE → EL]DUUH (le Z devient «] »)

- **3.c. Problème.** Les lettres proches de la fin de l'alphabet (X, Y, Z) débordent au-delà de Z (code 90) : on obtient des caractères qui ne sont *pas* des lettres ([, \,]). Il manque l'**enroulement** de l'alphabet (Z devrait redonner C, $Y \rightarrow B$, $X \rightarrow A$). De plus, une espace serait codée « # » ($32 + 3 = 35$).

- **4. Script corrigé** (on enroule l'alphabet pour X, Y, Z, et on laisse les non-lettres inchangées) :

```

1 def code_cesar(message):
2     message = message.upper()
3     coder = ''
4     for caractere in message:
5         n = ord(caractere)
6         if n < 65 or n > 90:
7             n_code = n
8         elif n > 87:
9             n_code = n + 3 - 26
10        else:
11            n_code = n + 3
12            lettre_c = chr(n_code)
13            coder = coder + lettre_c
14    return coder

```

- Avec ce script, ZYGOTE → CBJRWH (en effet $Z \rightarrow C$, $Y \rightarrow B$, $G \rightarrow J$, $O \rightarrow R$, $T \rightarrow W$, $E \rightarrow H$).

- **5.a. Décoder.** Pour décoder un message décalé de +3, il faut faire l'opération inverse : **retirer 3** au code ASCII (décalage de -3), en enroulant l'alphabet dans l'autre sens ($A \rightarrow X$, $B \rightarrow Y$, $C \rightarrow Z$).

- **5.b.** En décalant chaque lettre de 3 rangs en arrière :

EUDYR YRXV HWHV IRUW → **BRAVO VOUS ETES FORT.**